



《编译原理》

第5讲: RISC-V汇编

何冬杰
重庆大学

Assembly Language, ISAs

❑ Basic job of a CPU: execute lots of **instructions**.

- CPU: Centralized Processing Unit

❑ Instructions are the primitive operations that the CPU may execute.

- Like a sentence: **operations** (verbs) applied to **operands** (objects)

❑ An Instruction Set Architecture (ISA) defines what operations a particular CPU supports, and how it implements them.

- Assembly language: the low-level CPU instructions

- Machine language: how the instructions are represented, in bits

- Fundamental architectural features and design

❑ RISC-V ISA defines instructions for CPU down to the bit level:

add x18, x19, x10
assembly code



00000000101010011000100100110011
machine code



ISAs: RISC vs. CISC

□ Different CPUs implement different ISAs:

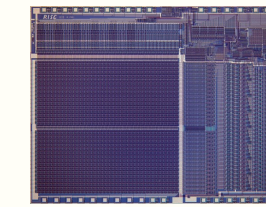
- ARM (cell phones), Intel x86 (i9, i7, i5), IBM/Motorola PowerPC (old Macs), MIPS, RISC-V, ...
- Early trend was to add more and more instructions to new CPUs to do elaborate operations.
 - VAX architecture had an instruction to multiply polynomials!

□ CISC: *Complex Instruction Set Architecture*, e.g., x86

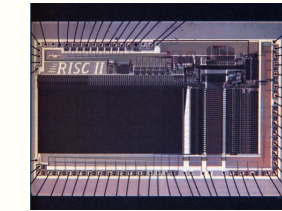
□ *Reduced Instruction Set Computer* (RISC) philosophy:

- Cocke IBM, **Patterson, Hennessy**, 1980s.
- Keep the instruction set small and simple. Let software do complicated operations by composing simpler ones.
- **Makes it easier to build fast hardware**: Laptops, Servers, IoT, ...
- This went against the conventional wisdom of the time.
(he who laughs last, laughs best)

RISC-V Architecture



RISC-I



RISC-II



❑ RISC-V Origin

- Lineage can be traced to RISC-I/II projects (1980s)
- Started in Summer 2010 to support open research and teaching at UC Berkeley
- As the project matured, it migrated to RISC-V foundation (www.risc-v.org)

❑ New (2015) open-source, license-free ISA spec

- Supported by growing shared software ecosystem
- Appropriate for all levels of computing system, from microcontrollers to supercomputers
- 32-bit, 64-bit, and 128-bit variants (class/textbook uses 32-bit)

❑ Why **RISC-V** instead of Intel **x86-64**?

- RISC-V is simple and elegant. We don't need to get bogged down in gritty details.
- RISC-V has exponential adoption, from microcontrollers to CPUs to warehouse-scale computers.
- Many commercial and research projects based on RISC-V.
 - Open-source and proprietary
- Widely used in education

RISC-V Base ISAs and Extensions

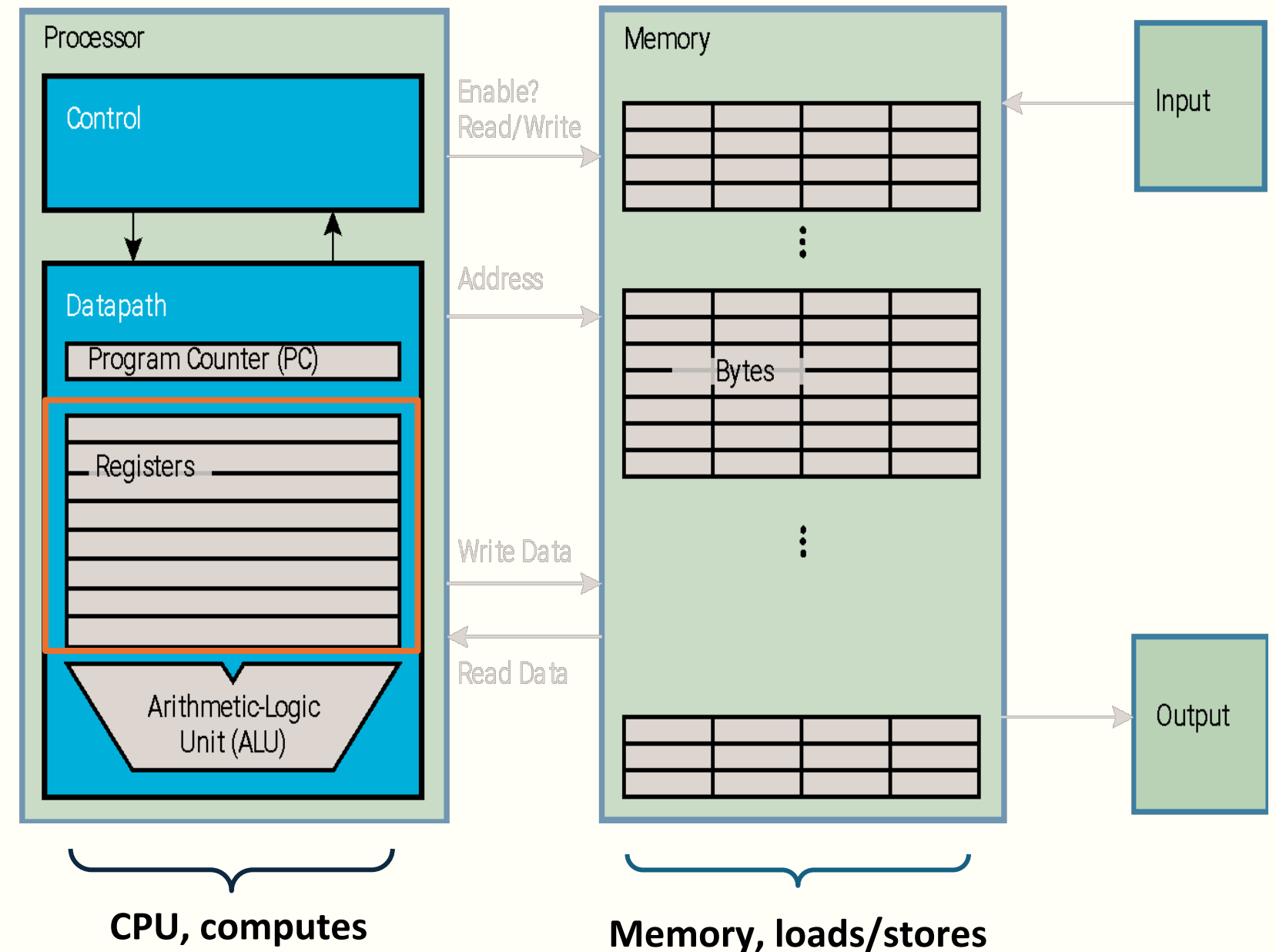
□ RISC-V 并非单一指令集，而是一系列逐层扩展的指令集

Name	Description
ISA base versions	
RV32	32-bit ISA
RV32E	32-bit ISA (embedded version)
RV64	64-bit ISA
RV64E	64-bit ISA (embedded version)
RV128	128-bit ISA
Extensions	
E	embedded version
I	base integer ISA
M	integer multiplication/division instructions
A	atomic instructions
F	single-precision floating point
D	double-precision floating point
V	vector instructions

本门课只涉及**RV64I+M**扩展，
即64 位基础整数指令集 +
整数乘法扩展。

Registers and Memory

- ❑ Computers have 2 main parts:
 - **CPU**, responsible for computing
 - Main **memory**, responsible for long-term data storage
- ❑ CPU is designed to be **fast**.
 - 3.33GHz: 0.3ns instr/second.
(speed of light: 10cm/0.3ns)
 - Main memory: takes ~100ns!
- ❑ CPU therefore stores a small amount of data through **registers**.



Registers (1ns) are inside the processor and faster than memory (100ns).

The Program Counter Register

- ❑ The program counter **pc** holds the address of the current instruction
- ❑ Program execution:
 - Fetch the instruction from memory using the PC
 - Execute instruction
 - Update PC to the next instruction
 - Normal instructions update PC to PC + 4 bytes
 - Branch instructions update PC according to the labels or target address
- ❑ No Instructions can use pc register directly.

RISC-V Register Names and Numbers

□ 32 registers in RISC-V

- Each ISA has a predetermined number of registers.

- Why? Registers are built into hardware.

- Each RISC-V register is 64 bits wide (in RV64)

- Groups of 64 bits are called words in RV64

□ Register Names and Mnemonics

- x0 is always 0

- Only 31 can hold values

□ See *RiscV.ml*

Register	Mnemonic	Description
$x0$	zero	Hard-wired to zero
$x1$	ra	Return address
$x2$	sp	Stack pointer
$x3$	gp	Global pointer
$x4$	tp	Thread pointer (thread-local storage)
$x5 - 7$	t0-2	Temporary registers
$x8$	s0/fp	Saved register/frame pointer
$x9$	s1	Saved register
$x10 - 11$	a0-1	Function arguments/return values
$x12 - 17$	a2-7	Function arguments
$x18 - 27$	s2-11	Saved registers
$x28 - 31$	t3-6	Temporary registers

RISC-V Assembly Language

□ Programs in Assembly language consist of

- **Instructions:** e.g., add x5, x6, x7

- May include pseudo-instructions
- Each instruction occupies one line.

- **Labels:** essentially the names of some addresses

- E.g., “loop:”, “main:”, “exit:”

- **Assembler Directives:** used by the assembler, not the CPU

- .text, .data, .global main, .word 10, .string “Hello”
- Different assemblers may have different directives

- **Comments:** written using the # symbol

- Anything written after a # on a line gets ignored

RISC-V assembly = Instructions + Labels + Directives (+ Comments)

RISC-V Instructions

□ Moving Values to Registers

Semantics	Example	Explanation
addi rd, rs1, imm	addi x1, x0, 5	$x1 \leftarrow 0 + 5$
add rd, rs1, rs2	add x1, x2, x3	$x1 \leftarrow x2 + x3$

□ Pseudoinstructions

- Not a real instruction; it uses existing instructions to produce equivalent behaviour.
- The assembler translates pseudoinstructions into real instructions.

Semantics	Example	Explanation
lui rd, imm	lui x1, 5	$x1 \leftarrow 5 \ll 12$
li rd, imm	li x1, 0xABCD1234	$x1 \leftarrow 0xABCD1234$

li x1 5 → **addi x1 x0 5**

RISC-V Instructions

□ Load and Store Instructions

Semantics	Example	Explanation
lw rd, imm(rs1)	lw x1, 32(sp)	$x1 \leftarrow \text{mem}[\text{sp} + 32]$
sw rs2, imm(rs1)	sw x1, 32(sp)	$\text{mem}[\text{sp} + 32] \leftarrow x1$
la rd, label	la x1, pi	$x1 \leftarrow \text{address}(\text{pi})$

lb, lh, lw, ld指令从对应内存从低位开始读取 1、2、4、8字节并符号扩展至目标寄存器。

lbu, lhu, lwu指令从对应内存从低位开始读取 1、2、4字节并零扩展至目标寄存器。

sb, sh, sw, sd指令往对应内存存储寄存器低位开始的1、2、4、8字节

RISC-V Instructions

□ Arithmetic instructions

Semantics	Example	Explanation
add rd, rs1, rs2	add x1, x2, x3	$x1 \leftarrow x2 + x3$
addi rd, rs1, imm	addi x1, x2, 5	$x1 \leftarrow x2 + 5$
sub rd, rs1, rs2	sub x1, x2, x3	$x1 \leftarrow x2 - x3$

Semantics	Example	Explanation
mul rd, rs1, rs2	mul x1, x2, x3	$x1 \leftarrow x2 \times x3$
div rd, rs1, rs2	div x1, x2, x3	$x1 \leftarrow x2 / x3$
rem rd, rs1, rs2	rem x1, x2, x3	$x1 \leftarrow \text{rem}(x2 / x3)$

addw, addiw, subw 只计算低32 bits, 然后符号扩展至64位

乘法会得到128位的数, mul保留低64位数, mulh得到高64位数

mulw, divw, divuw, remw, remuw也是只计算低32 bits, 然后符号扩展至64位

mulhu, divu, remu视操作数位unsigned; mulhsu视两个操作数一个是signed, 一个是unsigned

RISC-V Instructions

□ Logical and Shift Instructions

Semantics	Example	Explanation
and rd, rs1, rs2	and x1, x2, x3	$x1 \leftarrow x2 \text{ AND } x3$
andi rd, rs1, imm	andi x1, x2, 6	$x1 \leftarrow x2 \text{ AND } 6$
or rd, rs1, rs2	or x1, x2, x3	$x1 \leftarrow x2 \text{ OR } x3$
ori rd, rs1, imm	ori x1, x2, 9	$x1 \leftarrow x2 \text{ OR } 9$
xor rd, rs1, rs2	xor x1, x2, x3	$x1 \leftarrow x2 \text{ XOR } x3$
xori rd, rs1, imm	xori x1, x2, 7	$x1 \leftarrow x2 \text{ XOR } 7$
sll rd, rs1, rs2	sll x1, x2, x3	$x1 \leftarrow x2 \ll x3$
srl rd, rs1, rs2	srl x1, x2, x3	$x1 \leftarrow x2 \gg x3$
sra rd, rs1, rs2	sra x1, x2, x3	$x1 \leftarrow x2 \ggg x3$
slli rd, rs1, imm	slli x1, x2, 3	$x1 \leftarrow x2 \ll 3$
srli rd, rs1, imm	srli x1, x2, 3	$x1 \leftarrow x2 \gg 3$
srai rd, rs1, imm	srai x1, x2, 3	$x1 \leftarrow x2 \ggg 3$

sllw, srlw, sraw, slliw, srliw, sraiw指令只对低32位进行移位, 的到结果后符号扩展至64位

RISC-V Instructions

❑ Set-less-than (slt) Instruction

Semantics	Example	Explanation
slt rd, rs1, rs2	slt x1, x2, x3	if ($x2 < x3$) set x1 to 1
slti rd, rs1, imm	slti x1, x2, 5	if ($x2 < 5$) set x1 to 1
sltu rd, rs1, rs2	sltu x1, x2, x3	if ($x2 <_{\text{unsigned}} x3$) set x1 to 1
sltui rd, rs1, imm	sltui x1, x2, 5	if ($x2 <_{\text{unsigned}} 5$) set x1 to 1

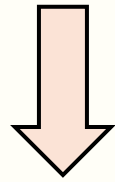
❑ Branch Instructions

Semantics	Example	Explanation
beq rs1, rs2, label	beq x1, x2, .foo	Branch to the . foo label if $x1 = x2$
bne rs1, rs2, label	bne x1, x2, .foo	Branch to the .foo label if $x1 \neq x2$
bge rs1, rs2, label	bge x1, x2, .foo	Branch to the .foo label if $x1 \geq x2$
blt rs1, rs2, label	blt x1, x2, .foo	Branch to the .foo label if $x1 < x2$
bgeu rs1, rs2, label	bgeu x1, x2, .foo	Similar to bge, considers unsigned values.
bltu rs1, rs2, label	bltu x1, x2, .foo	Similar to blt, considers unsigned values.

Lowering loops to Assembly

do-while

```
do {  
    line1;  
    line2;  
} while(cond)  
line3;
```



```
Loop:  
    line1;  
    line2;  
    if(cond) goto Loop;  
  
line3;
```

while

```
while(cond) {  
    line1;  
    line2;  
}  
line3;
```



```
Loop:  
    if(!cond) goto AfterLoop;  
    line1;  
    line2;  
    goto Loop;  
AfterLoop:  
    line3;
```

```
for(startline;cond;incline) {  
    line1;  
}  
line2;  
  
startline;  
while(cond) {  
    line1;  
    incline;  
}  
line2;
```



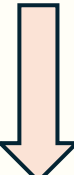
```
startline;  
Loop:  
    if(!cond) goto AfterLoop  
    line1;  
    incline;  
    goto Loop  
AfterLoop:  
    line2;
```



for

Example: Lowering loops to Assembly

```
int a = 0;
for(int i = 0; i < 10; i++) {
    if(i == 7) {
        break;
    }
    a = a + i;
}
a = a + 50;
```



```
int a = 0;
int i = 0;
Loop:
    int j = 10;
    if(i >= j) goto End;
    j = 7;
    if(i == j) goto End;
    a = a + i;
    i = i + 1;
    goto Loop;
End: a = a + 50;
```



```
Loop:
    li x10 0          #int a = 0;
    li x5 0           #int i = 0;

    li x6 10          #int j = 10;
    bge x5 x6 End      #if(i >= j) goto End;
    li x6 7           #j = 7;
    beq x5 x6 End      #if(i == j) goto End;
    add x10 x10 x5      #a = a + i;
    addi x5 x5 1        #i = i + 1;
    j Loop             #goto Loop;

End:
    addi x10 x10 50    #a = a + 50;
```

RISC-V Instructions

□ Unconditional Branches (mainly used in **function calls**)

- jal rd Label: $R[rd] = PC + 4$, $PC = \text{<address of Label>}$
 - j Label: $PC = \text{<address of Label>}$, is a pseudoinstruction, jal x0 Label
- jalr rd rs1 imm: $R[rd] = PC + 4$, $PC = R[rs1] + \text{imm}$
 - jr rs1: $pc = R[rs1]$, is a pseudoinstruction: jalr x0 rs1 0

Semantics	Example	Explanation
jal rd, label	jal x1, func	Jump to the func label and store the return address in $x1$
jalr rd, rs1, offset	jalr x1, x2, 20	Jump to the address $x2 + 20$ and store the return address in $x1$

Load Upper Immediates

❑ Load upper immediate

- lui rd imm: $\text{imm} = \text{imm} \ll 12$; $R[\text{rd}] = \text{imm}$
- Write an upper immediate to register rd

❑ Add upper immediate to PC

- Auipc rd imm: $\text{imm} = \text{imm} \ll 12$; $R[\text{rd}] = \text{pc} + \text{imm}$
- Add an upper immediate to the PC

Example: Function Calls

```
0x 0 main:
      li a0 4
      jal ra factorial
      8      addi a1 a0 0
      ...
     20      li a0 3
     24      jal ra factorial
     28      addi a1 a0 0
     ...
    48 factorial:
          addi t0 x0 1
     4c          addi t1 x0 1
    50 loop:
          bgt t0 a0 done
     54          mul t1 t1 t0
     58          addi t0 t0 1
     5c          j loop
    60 done:
          add a0 t1 x0
    64      jr ra
```

Function call:
jal @ 0x4

Function return:
jr @ 0x64

Function call:
jal @ 0x24

Function return:
jr @ 0x64

pc

0x4

 0x48
ra

0xABCD ABCD

 0x8

pc

0x64

 0x8
ra

0x8

pc

0x24

 0x48
ra

0xA

 0x28

pc

0x64

 0x28
ra

0x28

```
int factorial(int n) {
    int prod = 1;
    for (int i = 1; i <= n; i++) {
        prod *= i;
    }
    int retval = prod;
    return retval;
}
```

a0 is a register name for register **x10** and is used for **both**

- the first argument to a function, *and*
- the return value of a function.

Stack frame in RISC-V

❑ A function can choose to use a stack frame by manipulating **sp**:

- After a function is called, **create** a stack frame/**decrement** **sp**.
- When the function wraps up, **delete** the stack frame/**increment** **sp**.

❑ A stack frame can include:

- Space for local variables (e.g., stack arrays)
- Return “instruction” address (why?)
- Parameters (arguments) beyond the 8 argument registers

Example: Stack Frame in RISC-V

```

0x 0  main:
4      addi sp sp -12
...
...    x calls fooA
...
20     addi sp sp 12
24     j exit

28  fooA:
      addi sp sp -8
...
50     addi sp sp 8
54     jr ra
...
70  exit:
...

```

	+3	+2	+1	+0
0xFFFF FFD0	"main's" stack frame			
0xFFFF FFD8				
0xFFFF FFDC				
0xFFFF FFD4				
0xFFFF FFD0	0xB0BA CAFE			
0xFFFF FFCC	0xABCD EF12			
0xFFFF FFC8	0x3456 7890			
0xFFFF FFC4	0xB0BA CAFE			
0xFFFF FF	0xABCD EF12			

sp 0xFFFF FFD4

↙

"main's" stack pointer

Example: Stack Frame in RISC-V

		+3	+2	+1	+0
0x 0	main:	0xFFFF	FFDC	"main's" stack frame	
4	addi sp sp -12	0xFFFF	FFD8		
...	...	0xFFFF	FFD4		
...	x calls fooA	0xFFFF	FFD0	"fooA's" stack frame	
20	addi sp sp 12	0xFFFF	FFCC		
24	j exit	0xFFFF	FFC8	0x3456 7890	
28	fooA:	0xFFFF	FFC4	0xB0BA CAFE	
...	addi sp sp -8	0xFFFF	FF	0xABCD EF12	
50	...				
54	addi sp sp 8				
...	jr ra				
...	...				
70	exit:				
...	...				

sp 0xFFFF FFCC

↙

"fooA's" stack pointer

Example: Stack Frame in RISC-V

		+3	+2	+1	+0
0x 0	main:	0xFFFF	FFDC	"main's" stack frame	
4	addi sp sp -12	0xFFFF	FFD8		
...	...	0xFFFF	FFD4		
...	x calls fooA	0xFFFF	FFD0	"fooA's" stack frame	
20	addi sp sp 12	0xFFFF	FFCC		
24	j exit	0xFFFF	FFC8	0x3456 7890	
28	fooA:	0xFFFF	FFC4	0xB0BA CAFE	
...	addi sp sp -8	0xFFFF	FF	0xABCD EF12	
50	...				
54	addi sp sp 8				
...	jr ra				
...	...				
70	exit:				
...	...				

sp 0xFFFF FFD4

Restore the caller's (here, main's)
stack pointer before "returning"

Example: Stack Frame in RISC-V

			+3	+2	+1	+0
0x 0	main:		0xFFFF	FFDC	"main's" stack frame	
4	addi sp sp -12					
...	...		0xFFFF	FFD8		
...	x calls fooA		0xFFFF	FFD4		
20	addi sp sp 12		0xFFFF	FFD0	0xB0BA	CAFE
24	j exit		0xFFFF	FFCC	0xABCD	EF12
28	fooA:		0xFFFF	FFC8	0x3456	7890
	addi sp sp -8		0xFFFF	FFC4	0xB0BA	CAFE
...	...		0xFFFF	FF	0xABCD	EF12
50	addi sp sp 8					
54	jr ra					
...	...					
70	exit:					
...	...					

sp 0xFFFF FFD4

When main "has control", it sees the right stack pointer

Register Calling Convention

❑ Functions share the **same registers** and **calling conventions**.

- **Caller**: the calling function
- **Callee**: the function being called
- When the **callee** returns from executing, the **caller** needs to know:
 - Which registers may have changed, and which are guaranteed to be unchanged
- Register **calling conventions**:
 - A set of generally accepted rules as to which registers will be unchanged after a procedure call (jal) and which may be changed.

❑ RISC-V divides registers into two categories to reduce register spilling:

- **Caller-saved**: not preserved across calls
- **Callee-saved**: restored by the callee if modified

RISC-V Calling Convention

```
x0: zero  
x3: gp (global pointer)  
x4: tp (thread pointer)
```

❑ Caller-saved: (if data in them)

- Arguments a0-a7 (x10-x17) /return values (a0-a1)
- Return address, ra (x1)
- Temporary registers, t0-t6 (x5-7, x28-31)
- May not be preserved across function calls!
- Caller is responsible for saving these values for itself before calling a callee
 - 若 caller 在调用 callee 后还需要这些寄存器原来的值, 那么 caller 负责在调用前保存
 - To save register values, save to s0-s11 or to its stack frame

❑ Callee-saved: (if it uses them)

- Stack pointer sp (x2): has the address of the bottom of the current stack frame
- Saved registers, s0-s11 (x8-9, x18-27)
- Callee is responsible for saving/restoring these register values

Example: Recursive Factorial

❑ Caller-saved: a0, t0, ra

❑ Callee-saved: s0, sp

```
int factorial(int n) {  
    if (n == 1)  
        return 1;  
    return n * factorial(n - 1);  
}  
  
int main(void) {  
    // result is 6  
    int result = factorial(3);  
    // ...  
    return 0;  
}
```



```
main:  
    li a0 3  
    jal ra factorial  
    ...  
factorial:  
    addi sp sp -8  
    sw ra 0(sp)  
    sw s0 4(sp)  
    mv s0 a0  
    li t0 1  
    bne s0 t0 recurse  
    li a0 1  
    j epilogue  
recurse:  
    addi a0 s0 -1  
    jal ra factorial  
    mul a0 s0 a0  
epilogue:  
    lw ra 0(sp)  
    lw s0 4(sp)  
    addi sp sp 8  
    jr ra
```

Prologue

Epilogue

Fundamental Steps in Calling a Function

❑ Set up for callee:

- Save caller-saved registers if needed, either by copying to s0-s11 or to stack
- put arguments in a0, ..., a7, etc.

❑ Transfer control to function: `jal ra fnLabel`

❑ Prologue:

- Push stack frame/decrement sp
- Save callee-saved registers if will use s0-s11
- Save ra if calling subroutine
- Reserve space for local variables stored in memory (e.g., arrays).

❑ Perform the desired function task

❑ Epilogue:

- Put return value a0 (and a1 if necessary)
- Restore any modified callee-saved registers; restore ra if called subroutine

❑ Return control to point of origin: `j ra`

- Restore caller-saved registers if needed

```
main:
    li a0 3
    jal ra factorial
    ...
factorial:
    addi sp sp -8
    sw ra 0(sp)
    sw s0 4(sp)
    mv s0 a0
    li t0 1
    bne s0 t0 recurse
    li a0 1
    j epilogue
recurse:
    addi a0 s0 -1
    jal ra factorial
    mul a0 s0 a0
epilogue:
    lw ra 0(sp)
    lw s0 4(sp)
    addi sp sp 8
    jr ra
```

Prologue

Epilogue

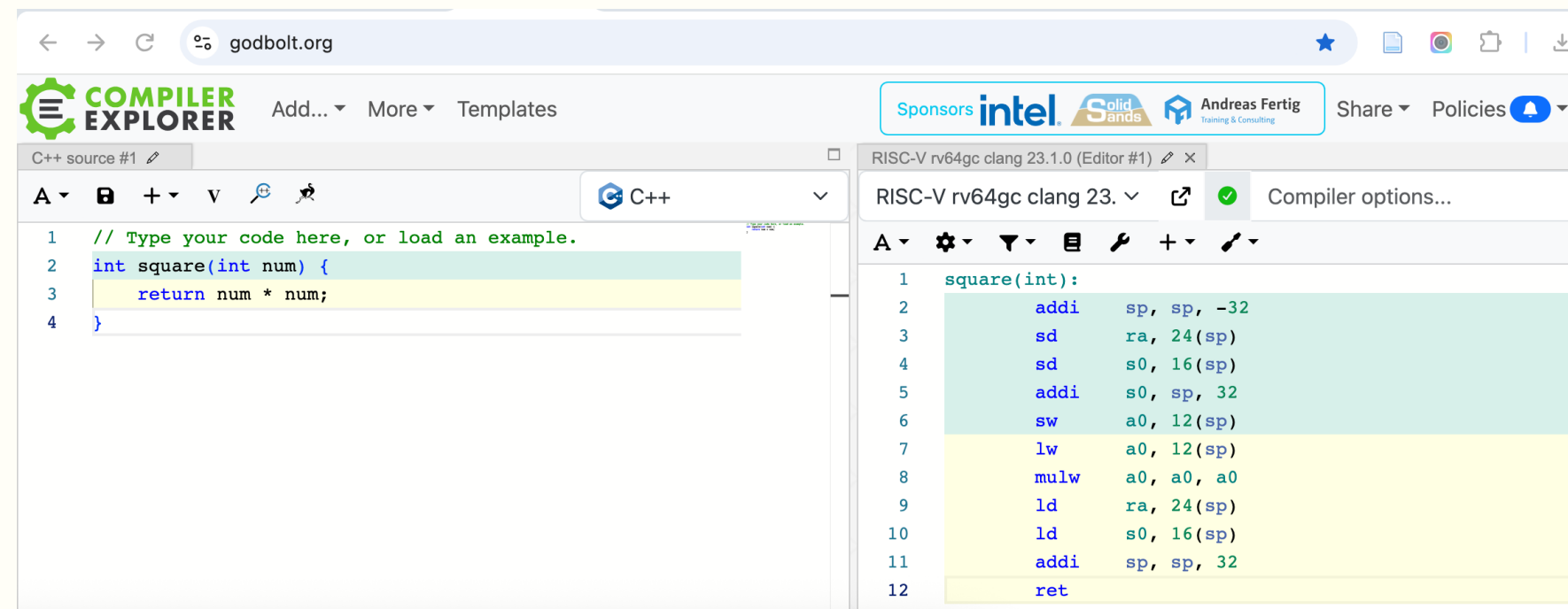
作业：熟悉RISC-V汇编指令

□利用学习的RV64I+M指令，完成下面两个程序的编写：

- 判断闰年：<http://compiler.plc-cqu.cn/#/oj?id=1693407735>
- 计算组合数：<http://compiler.plc-cqu.cn/#/oj?id=1693407736>

□现有工具如Compiler Explorer

- 支持将高级语言如C++程序转成RISC-V汇编
- 建议大家手动完成上述题目，然后对比自动生成的汇编，比较有何不同



The screenshot shows the Compiler Explorer interface on godbolt.org. On the left, the C++ source code is displayed in a text editor:

```
1 // Type your code here, or load an example.
2 int square(int num) {
3     return num * num;
4 }
```

On the right, the generated RISC-V assembly code for the same function is shown:

```
1 square(int):
2     addi    sp, sp, -32
3     sd      ra, 24(sp)
4     sd      s0, 16(sp)
5     addi    s0, sp, 32
6     sw      a0, 12(sp)
7     lw      a0, 12(sp)
8     mulw    a0, a0, a0
9     ld      ra, 24(sp)
10    ld      s0, 16(sp)
11    addi    sp, sp, 32
12    ret
```

Assembly vs Machine Code

- ❑ We write assembly language instructions

- e.g., “add x18, x19, x10”

- ❑ The machine interprets machine code bits

- e.g., “00000000101010011000100100110011”

- ❑ The **assembler** takes care of compiling assembly language to bits for us.

- It also provides a few conveniences

- E.g., Directives, Pseudoinstructions, labels,...

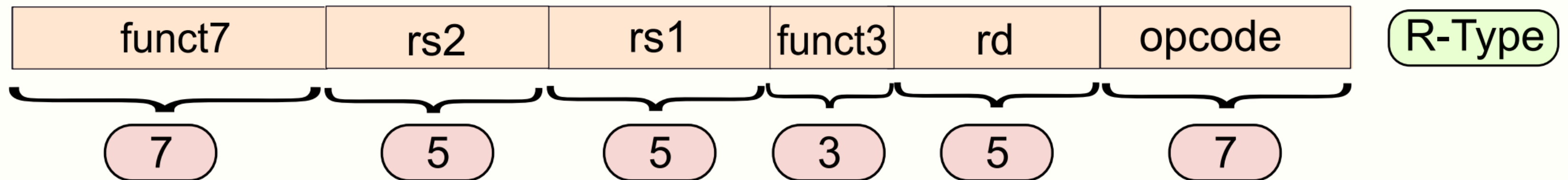
- ❑ Format of each RISC-V Instruction

- See *RiscV.ml*

RISC-V Instruction Formats

□ R-type Instruction Format

- A 5-bit unsigned integer [0-31] corresponds to a register number (x0-x31)
- For register-register arithmetic instructions: **add, sub, mul, div, rem, and, or, xor, sll, srl, sra, slt, sltu**
- **opcode**: determines what instruction format to use
- **funct3** and **funct7** determine what operation to perform



RISC-V Instruction Formats

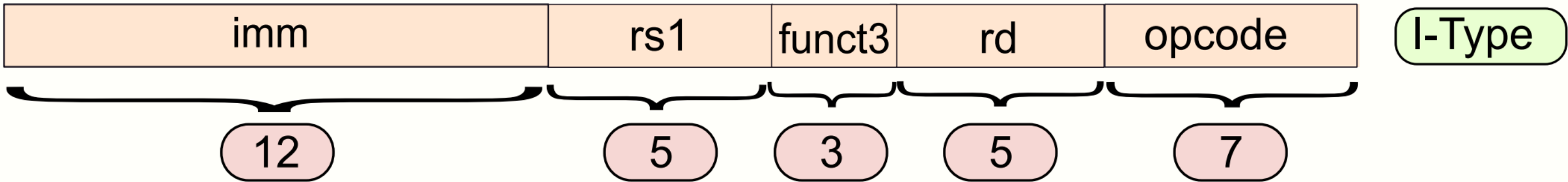
□ R-Type Instructions

funct7			funct3		opcode	
0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	111	rd	0110011	and
0000000	rs2	rs1	110	rd	0110011	or
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	001	rd	0110011	sll
0000000	rs2	rs1	101	rd	0110011	srl
0100000	rs2	rs1	101	rd	0110011	sra
0000000	rs2	rs1	010	rd	0110011	slt
0000000	rs2	rs1	011	rd	0110011	sltu

RISC-V Instruction Formats

I-type Instruction Format

- For register-immediate arithmetic instructions: addi, andi, ori, xori, slli, srli, srai, slti, sltiu, lw, jalr
- Imm[11:0] holds 12-bit wide immediate values: [-2048, +2047]
 - CPU sign-extends to 32 bits before using



I*-type

			funct3		opcode	
imm[11:0]		rs1	000	rd	0010011	addi
0000000	imm[4:0]	rs1	001	rd	0110011	slli
0000000	imm[4:0]	rs1	101	rd	0110011	srli
0100000	imm[4:0]	rs1	101	rd	0110011	srai

RISC-V Instruction Formats

□ I-Type instructions

			funct3		opcode	
imm[11:0]		rs1	000	rd	0010011	addi
imm[11:0]		rs1	111	rd	0010011	andi
imm[11:0]		rs1	110	rd	0010011	ori
imm[11:0]		rs1	100	rd	0010011	xori
0000000	imm[4:0]	rs1	001	rd	0010011	slli
0000000	imm[4:0]	rs1	101	rd	0010011	srli
0100000	imm[4:0]	rs1	101	rd	0010011	srai
imm[11:0]		rs1	010	rd	0010011	slti
imm[11:0]		rs1	011	rd	0010011	sltiu

Arithmetic, shift

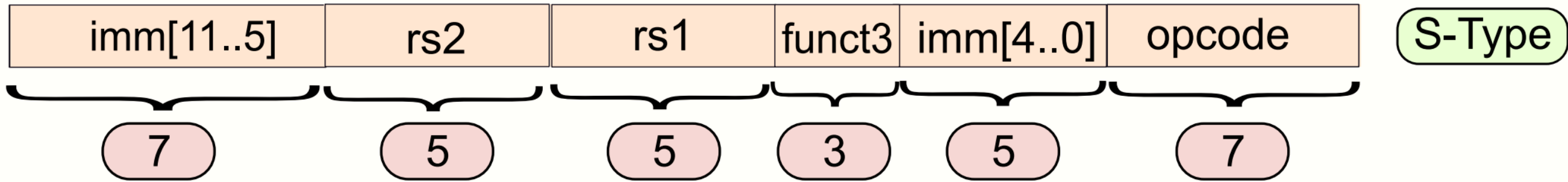
			funct3		opcode	
imm[11:0]		rs1	000	rd	0000011	lb
imm[11:0]		rs1	100	rd	0000011	lbu
imm[11:0]		rs1	001	rd	0000011	lh
imm[11:0]		rs1	101	rd	0000011	lhu
imm[11:0]		rs1	010	rd	0000011	lw
imm[11:0]		rs1	000	rd	1100111	jalr

Load, Jump

RISC-V Instruction Formats

□ S-Type Instruction Format

- For Store Instructions: sw, sh, sb
 - Store address = (Base Register) + (Immediate Offset)
- Why split up the immediate field in S-Type?
 - For simple hardware design!



□ S-Type Instructions:

			funct3		opcode	
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	sb
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	sh
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	sw

Exercise: How to encode “add x4 x3 x2”?

❑ Lookup table:

funct7			funct3		opcode	
0000000	rs2	rs1	000	rd	0110011	add

❑ Encoding:

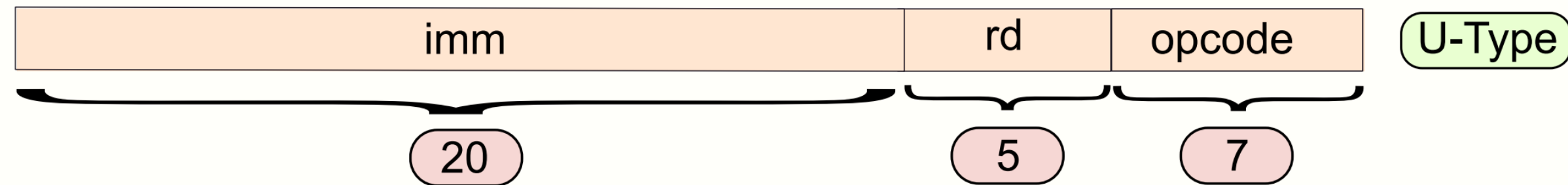
funct7			funct3		opcode	
0000000	00010	00011	000	00100	0110011	add

❑ Hex: 0x 00 21 82 33

RISC-V Instruction Formats

❑ U-Type Instruction Format

- For upper immediate instructions: lui, auipc



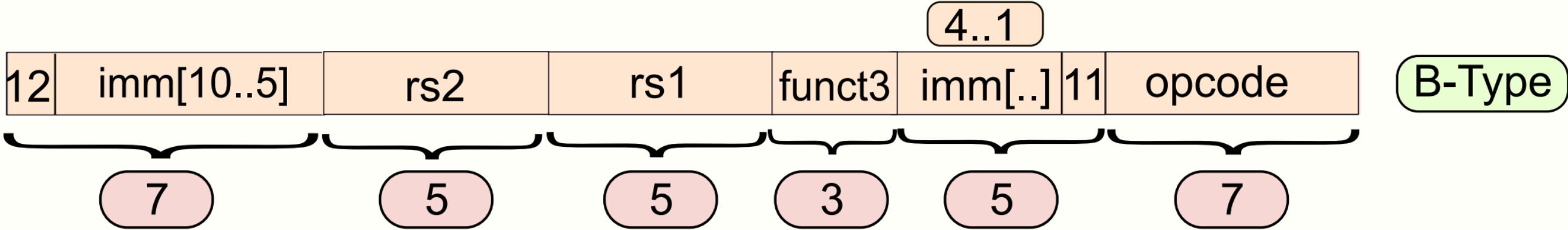
❑ U-Type Instructions

		opcode	
imm[31:12]	rd	0010111	auipc
imm[31:12]	rd	0110111	lui

RISC-V Instruction Formats

□ B-Type Instruction Format

- Offset = imm[12:0] is a 13-bit integer, imm[0] is always 0
 - PC relative offsets are $[-2^{12}, 2^{12})$ byte changes (how many instructions away?)



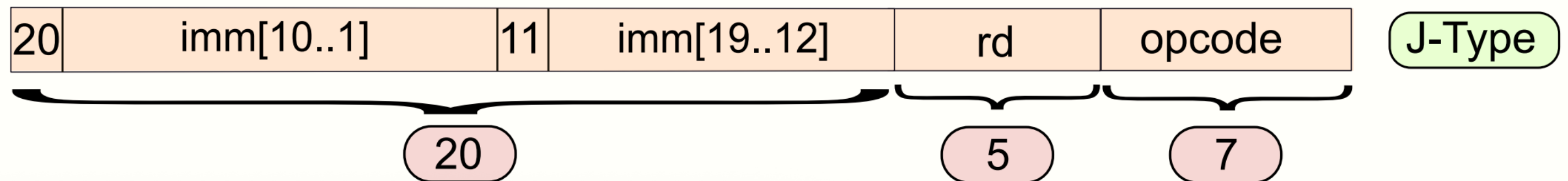
□ B-Type Instructions

			funct3	opcode		
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	beq
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	bne
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	blt
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	bltu
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	bge
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	bgeu

RISC-V Instruction Formats

□ J-Type Instruction Format

- Used only for jal
- Offset = imm[20:0] is a 21-bit integer (2's complement), imm[0] is always 0



□ B-Type Instruction



Assembler and Disassembler

❑ Assembler: translate assembly language into machine language

add x18, x19, x10  00000000101010011000100100110011
assembly code machine code

❑ Disassembler: translate machine language into assembly language

00000000101010011000100100110011  add x18, x19, x10
machine code assembly code

❑ Both are (simplified) compilers in principle.

❑ See *Assembler.ml*

汇编器原理

□ 汇编器本质上是个简化版的编译器

- 负责将汇编语言程序翻译成机器语言程序
- 汇编语言非常简单：以至于不需要介绍前端的词法和语法解析（见 [AsmUtil.ml](#)）
 - 每行对应一条指令，由操作符和若干操作数组成，各操作数之间以逗号分隔。
 - 汇编器指令（Directive）可以自己定义
 - 常见如 .text, .data, .bss, .globl, .quad, .byte, .zero, .asciz
 - 注释以 # 开头，简单起见，仅支持整行注释

RISC-V assembly = Instructions + Labels + Directives (+ Comments)

□ 汇编器将每条指令从文本形式转换为二进制形式

- 指令 format 定义了指令的编码和转化规则
- 指令中使用的 Label 需转换为相对偏移地址
- 指令中使用的全局数据需转换为绝对地址

汇编器原理：Two-Pass Assembler

□ Pass 1: 建立符号表 (Symbol Table)

- 从上到下扫描汇编程序，计算每条指令的地址 (Address)
 - RISC-V ISA指令每条占4字节，伪指令可能不止4字节
- 遇到Label，记录Label的地址 (Address)

```
address ← 0
SymbolTable ← {}
for each line in program:
    if line contains Label: SymbolTable[Label] ← address
    if line contains Instruction: address ← address + InstructionSize
    else if line contains Directive: address ← address + DataSize
return SymbolTable
```

汇编器原理：Two-Pass Assembler

□ Pass 2: 生成机器码 (Machine Code)

- 再次从上到下扫描，根据Instruction Format编码每条指令
 - 将Label查表，转换为对应地址或相对偏移
 - 找不到情况下，需记录relocation信息方便后续链接器 (linker回填)
 - 将全局符号解析为对应地址 (一般为绝对地址)

```
for each line in program:
    if line is an instruction:
        encode instruction using Instruction Format
        resolve symbols
        create Relocation Entry if needed
        emit machine code
    else if line is a directive:
        encode directive
        emit data if needed
emit Relocation Table
```

汇编器原理： 示例

```
ADD R1, R2, R3
BEQ R1, R0, LOOP
ADD R4, R4, R5
LOOP: SUB R6, R6, R1
```

Pass 1

```
0:      ADD R1, R2, R3
4:      BEQ R1, R0, LOOP
8:      ADD R4, R4, R5
12: LOOP: SUB R6, R6, R1
```

Symbol Table:
LOOP -> 12

Pass 2

Address	Assembly	Binary	Hex
0	ADD x1,x2,x3	00000000001100010000000010110011	0x003100B3
4	BEQ x1,x0,LOOP	000000000000000001000010001100011	0x00008463
8	ADD x4,x4,x5	000000000101001000000001000110011	0x00520233
12	LOOP: SUB x6,x6,x1	010000000001001100000001100110011	0x40130333

汇编器原理

□ 汇编程序中可能引用外部程序中的符号

- 外部函数、全局数据
- 汇编器通常用占位符记录下这些暂不能确定的引用，留待链接器解析后回填
- 链接器：有时间的话，留待最后一次课和大家交流

□ 当前Two-Pass Assembler可以改写为One-Pass Assembler

- **核心思想**（若大家能实现出来，有bonus!）：
- 从前往后扫描指令，计算指令地址，若指令用到label，
 - 若符号表已记录该label，则计算相对偏移，用于指令编码
 - 若符号表未记录该label，记录映射关系表：{label_x -> [addr1, addr2, ...]}
 - 表示addr_i这些地址有用到label_x
- 扫描过程中遇到label，记录到符号表中，同时根据映射关系表进行回填

汇编器原理：教学用汇编器 v.s. 真实汇编器

□教学用Assembler

- 我们假设每个汇编程序不引用外部标签
 - 因此不需要链接器 (Linker)
- 假设写的汇编程序都很小：
 - 也就是B-Type Instructions的12位立即数 (imm) 够用
 - 否则branch指令需要换成Jal指令
 - 如何判断一个label够不够远？指令大小不确定性！

BEQ x1, x2, FAR_AWAY



BNE x1, x2, NEXT
JAL x0, FAR_AWAY
NEXT:

□真实汇编器：

- 支持许多伪指令和汇编指令 (directives)
- 需要多次迭代，多个pass

目标文件格式

- 汇编器生成的机器码可以在裸机上执行
- 应用程序大多需要操作系统调度执行，需告知OS如何加载和执行
- 目标文件格式：
 - 定义了程序的二进制文件如何组织，使Linker和 OS Loader能理解和处理它
 - 负责组织如下信息：
 - Machine Code, Symbol Table, Relocation Information
 - Sections (.text、.data、.bss 等) , Debug Information 等

操作系统	主要目标文件格式	常见文件
Linux	ELF	.o, executable, .so
macOS / iOS	Mach-O	.o, executable, .dylib
Windows	PE (Portable Executable)	.obj, .exe, .dll

目标文件格式：以ELF格式为例

□ ELF主要用于描述Linux系统上可执行文件格式

□ 包括ELF 首部和紧跟其后的文件数据部分：

- See *ELF.ml*

□ 文件数据部分：

- 程序头表（Program header table）：

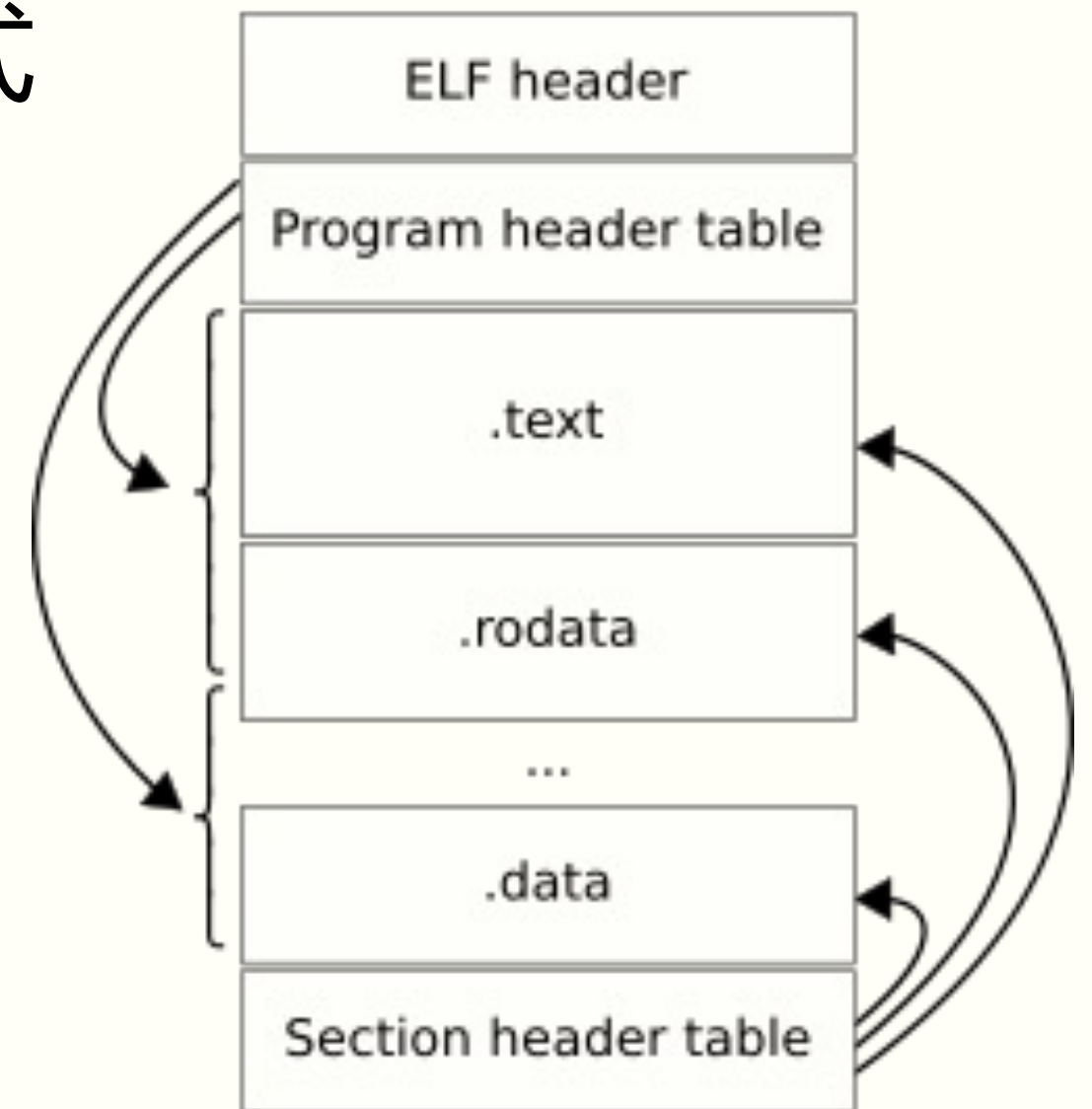
- 描述 0 个或多个内存段信息

- 分段头表（Section header table）：

- 描述 0 段或多段链接与重定位需要的数据

□ 更详细内容超过本课程范围

- <https://www.cs.cmu.edu/afs/cs/academic/class/15213-f00/docs/elf.pdf>



Lab2: using OCaml to write a RISC-V assembler

□支持将RV64I+M汇编程序翻译成二进制格式对象文件

References

❑ RISC-V Assembly Language

- <https://www.cse.iitd.ac.in/~srsarangi/archbook/chapters/riscv.pdf>

❑ Great Ideas in Computer Architecture

- By Dan Garcia at UC Berkeley. <https://cs61c.org/fa26/>

❑ ELF file format

- <https://www.cs.cmu.edu/afs/cs/academic/class/15213-f00/docs/elf.pdf>